

Übung Cybersicherheit

Übungsblatt 13: Reverse Engineering und Malware

Christian Niesler
Tristan Löding

Fakultät für Informatik
Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>
Universität Duisburg-Essen

Aufgabe 1

- a) *Benutzen Sie das Programm strings um Zeichenketten aus den Dateien 1, 2 und 3 zu extrahieren.
Um welche Programme handelt es sich bei den jeweiligen Dateien?
Was können Sie anhand der Zeichenketten außerdem über die Funktionsweise der Programme sagen?*

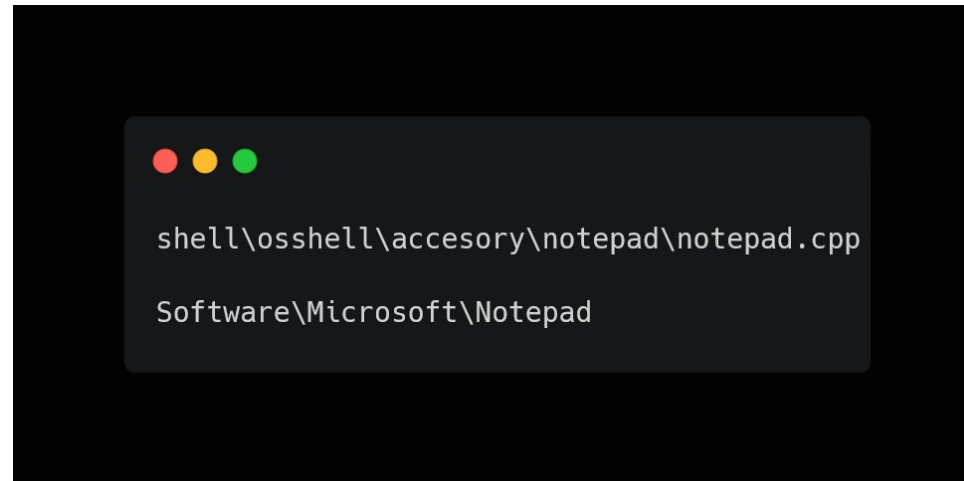
- 1. Datei: 7-Zip:
- Nützliche Strings:

```
Usage: 7z <command> [<switches>...] <archive_name> [<file_names>...] [@listfile]
<Commands>
  a : Add files to archive
  b : Benchmark
  d : Delete files from archive
  e : Extract files from archive (without using directory names)
  h : Calculate hash values for files
  i : Show information about supported formats
  l : List contents of archive
  rn : Rename files in archive
  t : Test integrity of archive
  u : Update files to archive
  x : eXtract files with full paths
```

Aufgabe 1

- a) *Benutzen Sie das Programm strings um Zeichenketten aus den Dateien 1, 2 und 3 zu extrahieren.
Um welche Programme handelt es sich bei den jeweiligen Dateien?
Was können Sie anhand der Zeichenketten außerdem über die Funktionsweise der Programme sagen?*

- 2. Datei: Notepad
- Nützliche Strings:

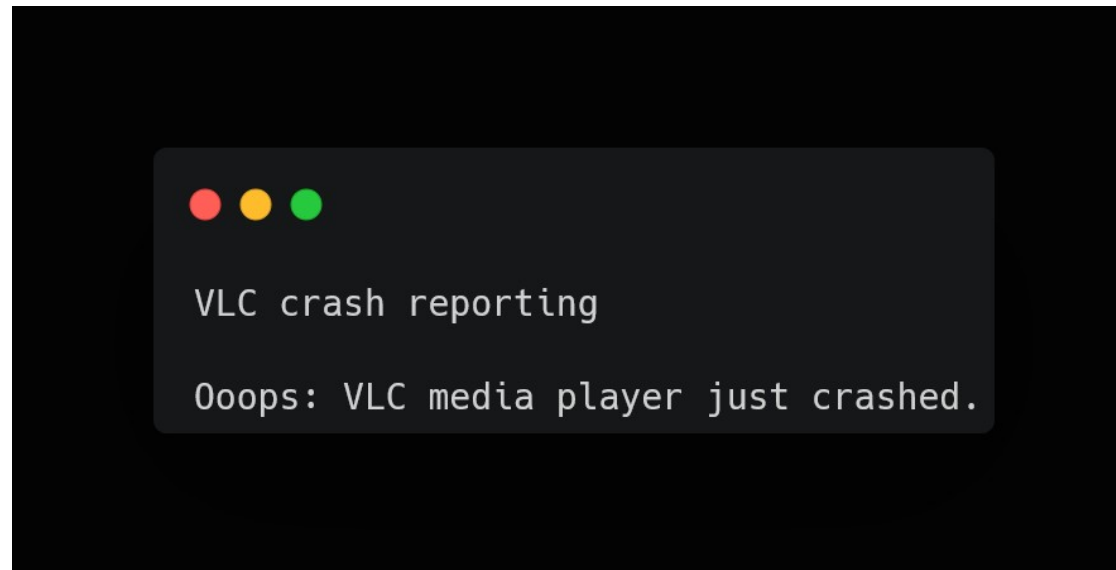


```
shell\osshell\acesory\notepad\notepad.cpp  
Software\Microsoft\notepad
```

Aufgabe 1

- a) *Benutzen Sie das Programm strings um Zeichenketten aus den Dateien 1, 2 und 3 zu extrahieren.
Um welche Programme handelt es sich bei den jeweiligen Dateien?
Was können Sie anhand der Zeichenketten außerdem über die Funktionsweise der Programme sagen?*

- 3. Datei: VLC
- Nützliche Strings:



Aufgabe 1

b) Die Ausgabe für jedes der drei Programme enthält viele Zeichenketten, welche keine sinnvolle Information enthält. Erläutern Sie wie es dazu kommt.

```
@.data
.pdata
@.rsrc
@.reloc
HSUVWATAUAVAWH
"u      D:
(A_A^A]A\_^[
HSUVWH
(HcY
(\_^[
HSUVWH
D$(H
T$HH
T$HH
T$(H
h\_^[
L$ SUVWATAUAVAWH
```

- Programme wie **strings** versuche **jegliche** Daten als Zeichenketten zu interpretieren
- Allerdings enthalten Programme auch andere Datentypen (etwa int oder double) und auch Code (Instruktionen)
- Manchmal können diese Daten auch als Zeichenketten interpretiert werden. Deswegen enthält die Ausgabe so viele Strings welche keinen offensichtlichen Sinn haben

Aufgabe 2

a) Analysieren Sie das Programm `mal_linux` bzw. `mal_windows.exe` und finden Sie heraus welcher Domainname aufgelöst werden soll.

Hinweis: Der Domainname der aufgelöst werden soll existiert nicht. Es wird eine entsprechende Fehlermeldung ausgegeben.

```
debug041:00EA2AB8 db 77h ; w
debug041:00EA2AB9 db 77h ; w
debug041:00EA2ABA db 77h ; w
debug041:00EA2ABB db 2Eh ; .
debug041:00EA2ABC db 65h ; e
debug041:00EA2ABD db 76h ; v
debug041:00EA2ABE db 69h ; i
debug041:00EA2ABF db 6Ch ; l
debug041:00EA2AC0 db 2Dh ; -
debug041:00EA2AC1 db 64h ; d
debug041:00EA2AC2 db 6Fh ; o
debug041:00EA2AC3 db 6Dh ; m
debug041:00EA2AC4 db 61h ; a
debug041:00EA2AC5 db 69h ; i
debug041:00EA2AC6 db 6Eh ; n
debug041:00EA2AC7 db 2Eh ; .
debug041:00EA2AC8 db 61h ; a
debug041:00EA2AC9 db 62h ; b
debug041:00EA2ACA db 63h ; c
debug041:00EA2ACB db 64h ; d
```

Domainname: **www.evil-domain.abcd**

Aufgabe 2

b) Extrahieren Sie den obfuskierten Domainnamen und reimplementieren Sie den Deobfuskiierungsalgorithmus, so dass Sie aus den extrahierten Daten den Domainname im Klartext anzeigen lassen können

```
Stack[00004810]:0061FEC0 db 16h : 22
Stack[00004810]:0061FEC4 db 12h : 18
Stack[00004810]:0061FEC8 db 12h : 18
Stack[00004810]:0061FECC db 37h : 55
Stack[00004810]:0061FED0 db 7Ch : 124
Stack[00004810]:0061FED4 db 63h : 99
Stack[00004810]:0061FED8 db 7Ch : 124
Stack[00004810]:0061FEDC db 7Dh : 125
Stack[00004810]:0061FEE0 db 3Ch : 60
Stack[00004810]:0061FEE4 db 71h : 113
```

```
Stack[00004810]:0061FEE8 db 7Ah : 122
Stack[00004810]:0061FEEC db 74h : 116
Stack[00004810]:0061FEF0 db 78h : 120
Stack[00004810]:0061FEF4 db 6Ch : 108
Stack[00004810]:0061FEF8 db 6Bh : 107
Stack[00004810]:0061FEFC db 2Fh : 47
Stack[00004810]:0061FF00 db 60h : 96
Stack[00004810]:0061FF04 db 67h : 103
Stack[00004810]:0061FF08 db 66h : 102
Stack[00004810]:0061FF0C db 7Dh : 125
```

Aufgabe 2

b) Extrahieren Sie den obfuskierten Domainnamen und reimplementieren Sie den Deobfuskiierungsalgorithmus, so dass Sie aus den extrahierten Daten den Domainname im Klartext anzeigen lassen können

Mit Hilfe eines Decompilers können die Sie herausfinden, dass einfaches Byte-XOR genutzt wird um den String zu verschleiern. Eine einfache Lösung in Python ist die folgende:

```
obfus_bytes = [22, 18, 18, 55, 124, 99, 124, 125, 60, 113, 122, 116, 120, 108, 107, 47, 96, 103, 102, 125]
key = [97, 101, 101, 25, 25, 21, 21, 17, 17, 21, 21, 25, 25, 5, 5, 1, 1, 5, 5, 25]
temp = ""
for i in range(len(obfus_bytes)):
    temp += chr(obfus_bytes[i] ^ key[i])
print("Deobfuscated string with key:", temp)
```


Aufbau des Stacks: Funktionsprolog und -epilog

Beispiel anhand des Codes aus Übung 9

```
1 int pw_check(char *password) {
2     int auth = 0;
3
4     // Allokiere Speicher auf dem Stack
5     char pw_buffer[16] = {0};
6
7     // Kopiere den *gesamten* String "password" nach "pw_buffer"
8     strcpy(pw_buffer, password);
9
10    // Falls das eingegeben Passwort "cyssec" ist,
11    // setze die Variable "auth"
12    if (strcmp(pw_buffer, "cyssec") == 0)
13        auth = 1;
14
15    return auth;
16 }
17
18 int main(int argc, char *argv[]) {
19
20     if (argc != 2) {
21         printf("Benutzung: %s <password>\n", argv[0]);
22         exit(0);
23     }
24
25     // Das eingegebene Passwort wird in der Variable "pw" gespeichert.
26     char *pw = argv[1];
27
28     // Die Funktion "pw_check" prüft das Passwort.
29     if (pw_check(pw)) printf("Gut gemacht!\n");
30     else printf("Zzzzzz...\n");
31 }
```

Was für Informationen (für den Aufbau des Stackframe) gewinnen wir aus dem Code:

- Es gibt ein Argument: (char *password)
- Zudem zwei lokale Variablen:
 - Einen Integer (4 Bytes)
 - Einen Buffer (16 Bytes)
- Klar ist: pw_check wurde von einer anderen Funktion aufgerufen

Funktionsprolog (x86)

Der Prolog einer Funktion wird zu Beginn der Funktion ausgeführt. Sein Hauptzweck ist es, den Stack-Frame für die Funktion vorzubereiten. Typischerweise sieht dieser so aus

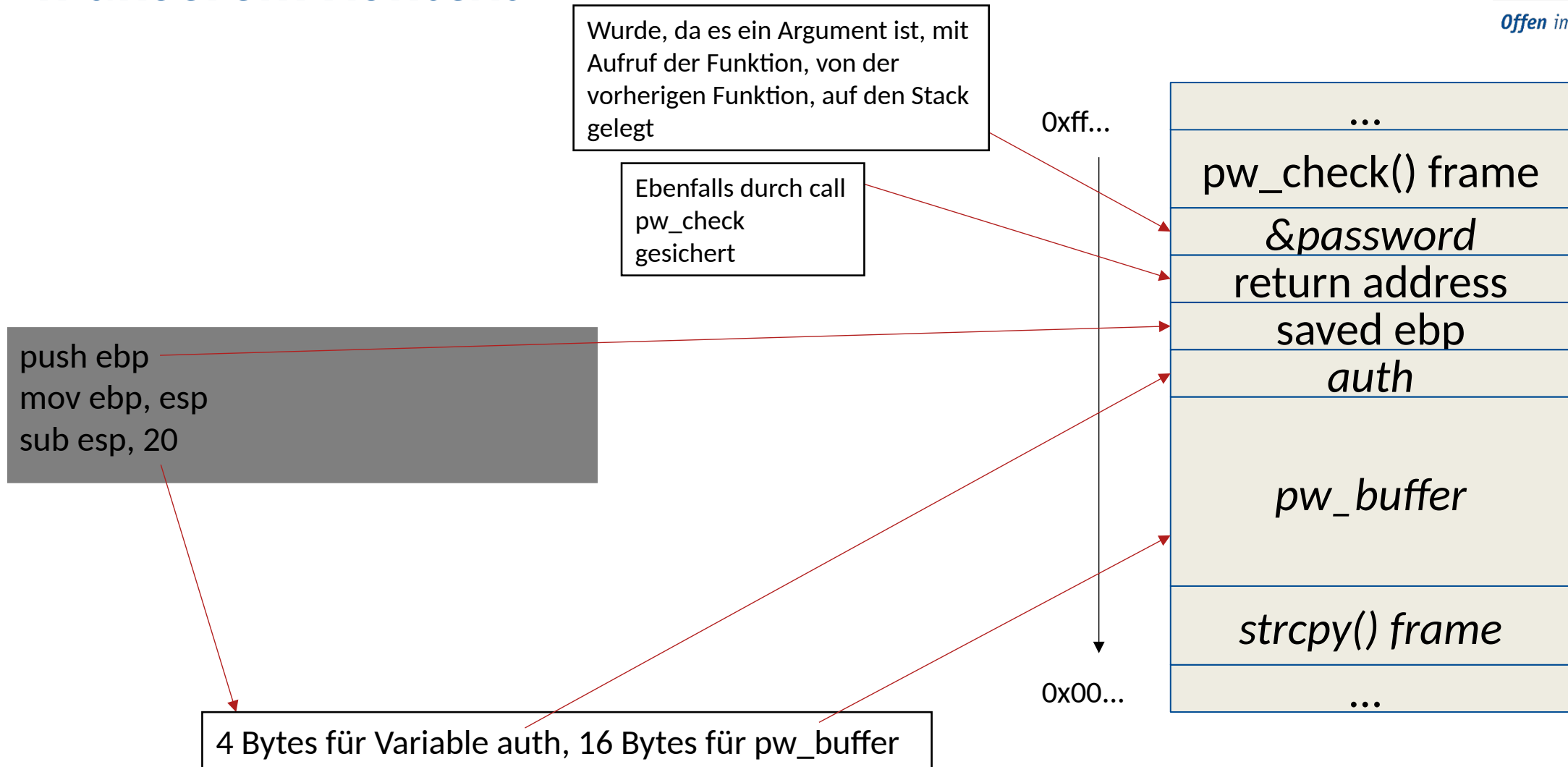
```
push ebp  
mov ebp, esp  
sub esp, <Größe der lokalen Variablen>
```

Legt den aktuellen Wert des Base-Pointers auf den Stack
-> Sichert Kontext der vorherigen Funktion

Setzt Base-Pointer auf den aktuellen Stack-Pointer um einen neuen Stack-Frame „starten“ zu können
-> ebp befindet sich damit am Beginn des neuen Stack-Frame, bei den lokalen Variablen und der Rücksprungadresse

Schafft innerhalb des neuen Stack-Frames Platz für lokale Variablen
-> Da ein Stack „nach unten“ wächst wird dekrementiert

In unserem Kontext



Funktionsepilog (x86)

Der Epilog ist ein Standardabschnitt am Ende einer Funktion, der dafür verantwortlich ist, den Zustand des Stacks zu bereinigen und die Funktion sauber zu verlassen, um zur aufrufenden Funktion zurückzukehren. Der Epilog stellt sicher, dass die durch den Prolog vorgenommenen Änderungen am Stack und an den Registern rückgängig gemacht werden, damit der ursprüngliche Kontext der aufrufenden Funktion wiederhergestellt wird.

```
mov esp, ebp  
pop ebp  
ret
```

Zurücksetzen des Stack-Pointers (esp) auf den Base-Pointer (ebp), damit alle lokalen Variablen, die Speicherplatz auf dem Stack belegen, effektiv "vergessen" werden.

Nimmt die Rücksprungadresse, die sich jetzt oben auf dem Stack befindet (nachdem pop ebp den Stack-Pointer um 4 Bytes erhöht hat), und springt zu dieser Adresse.

Entfernt den obersten Wert vom Stack (den alten ebp-Wert) und lädt ihn zurück in das ebp-Register.
-> Stack-Frame der aktuellen Funktion ist aufgelöst und ebp zeigt wieder auf den Base-Pointer der aufrufenden Funktion.

Übung Cybersicherheit SoSe 2025

Vielen Dank für Ihre Aufmerksamkeit

Christian Niesler
Tristan Löding

Fakultät für Informatik
Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi